

MEDIATEK

MT8668 Hypervisor OTA LLA User Manual

版本: 1.0
出版日期: 2026-01-28

Use of this document and any information contained therein is subject to the terms and conditions set forth in Exhibit 1. This document is subject to change without notice.

版本记录

版本	日期	作者	描述
1.0	2026-01-28	纪明	正式版

目录

版本记录 2

目录 3

图片目录 4

表格目录 5

1 Hypervisor (L+L+A) OTA 方案说明 6

2 Hypervisor (L+L+A)整体集中升级 7

2.1 启用整体集中升级方案 7

2.2 方案概述 7

2.2.1 基础功能 7

2.2.2 Hypervisor OTA 名词解释 8

2.3 架构/进程概述 9

2.3.1 Hypervisor (L+L+A) OTA 升级架构 9

2.3.2 Hypervisor (L+L+A) 升级包编译架构 10

2.4 如何编译 Hypervisor (L+L+A) OTA 升级包 11

2.4.1 编译 hypervisor_target_files.zip 11

2.4.2 编译 Hypervisor (L+L+A)全量升级包 12

2.4.3 编译 Hypervisor (L+L+A)差分升级包 12

2.5 升级包结构解析 13

2.6 如何进行 Hypervisor (L+L+A) OTA 升级 14

2.6.1 全包升级 (Normal Mode) 15

2.6.2 差分升级 (Normal Mode) 15

2.7 如何修改升级包的签名 16

2.8 如何删减/新增 OTA 升级分区 16

2.8.1 OTA 升级分区的来源 16

2.8.2 删减 OTA 升级分区 17

2.8.3 新增 OTA 升级分区 18

2.9 AB 系统的镜像加载选择 21

2.9.1 Boot Control 信息说明 21

2.9.2 镜像加载选择流程 22

2.10 系统回滚 23

2.10.1 系统回滚的基本原则	23
2.10.2 系统回滚的检测流程	25
2.10.3 系统回滚的 Boot Control 参数变化	25
2.10.4 系统回滚测试用例	27
2.11 断电升级保护	30
2.11.1 升级状态信息	30
2.11.2 Resume 升级检查流程	31
2.12 Hypervisor OTA API 说明	32
2.12.1 启动升级 (Normal Mode)	32
2.12.2 获取升级进度 (Normal Mode)	33
2.12.3 切换 Boot Slot (Normal Mode)	34
2.12.4 获取当前启动 Slot (Normal Mode)	35
2.12.5 中止升级 (Normal Mode)	35
2.12.6 Hypervisor OTA API 使用步骤	36
附件一 附加条款	37

图片目录

图 2-1. Hypervisor (L+L+A) OTA Update 架构	9
图 2-2. Hypervisor (L+L+A) build otapackage 架构	10
图 2-3. Hypervisor (L+L+A) otapackage 结构	13
图 2-4. payload.bin 组成结构	14
图 2-5. OTA 升级分区来源	16
图 2-6. Hypervisor (L+L+A) Boot Control 参数	21
图 2-7. Hypervisor (L+L+A) 镜像加载流程	22
图 2-8. OTA 升级后 Hypervisor 启动失败场景	23
图 2-9. OTA 升级后 Yocto 启动失败场景	24
图 2-10. OTA 升级后 Android 启动失败场景	24
图 2-11. OTA 升级后 Tbox 启动失败场景	24
图 2-12. Hypervisor(L+L+A) 回滚检测流程	25
图 2-13. Hypervisor & Yocto & Android & Tbox 启动状态	25
图 2-14. Hypervisor & Yocto & Android & Tbox 启动状态	26
图 2-15. Hypervisor & Yocto & Android & Tbox 启动状态	26
图 2-16. Hypervisor & Yocto & Android & Tbox 启动状态	27

图 2-17. Hypervisor & Yocto & Android & Tbox 启动状态.....27

图 2-18. 升级状态信息.....30

图 2-19. Hypervisor resume update 流程.....31

图 2-20. Hypervisor check resume update 流程.....31

表格目录

表 2-1. 名词解释.....8

表 2-2. 升级包组件描述.....13

表 2-3. payload.bin 组件描述.....14

表 2-4. Hypervisor (L+L+A) Boot control 参数说明.....21

表 2-5. 升级状态信息说明.....30

1 Hypervisor (L+L+A) OTA 方案说明

L+L+A 是指 Linux SOS (Yocto) + Linux UOS (Tbox) + Android UOS (Android OS)

对于 Hypervisor (L+L+A)的 OTA 升级, Mediatek 支持如下升级方案:

方案一: 整体集中升级, 即 Linux SOS (Yocto) 作为 Host OS 去统一升级 Linux SOS (Yocto) + Linux UOS (Tbox) + Android UOS (Android OS), 详细介绍请参考第 2 章 [Hypervisor \(L+L+A\)整体集中升级](#)。

2 Hypervisor (L+L+A)整体集中升级

本章节主要介绍 Hypervisor (L+L+A) OTA Update 整体集中升级方案，包括启用该方案的 config 配置、方案架构概述、升级包编译和系统回滚，此方案为公版默认启用方案。

2.1 启用整体集中升级方案

如果需要启用集中升级方案，请设定如下 config：

1. 在 LK2 的<yocto_project_name>.mk 和<uos_project_name>.mk 内添加 SUPPORT_BOOTCTRL_MULTI_OS := yes

```
e.g.,
src/bsp/lk2/project/auto8668p1_64_hyp.mk
SUPPORT_BOOTCTRL_MULTI_OS := yes
src/bsp/lk2/project/auto8668p1_64_hyp-an.mk
SUPPORT_BOOTCTRL_MULTI_OS := yes
src/bsp/lk2/project/auto8668p1_64_hyp_uos.mk
SUPPORT_BOOTCTRL_MULTI_OS := yes
```

2. 在 meta 的<yocto_project_name>.conf 和<uos_project_name>.conf 内添加 SUPPORT_BOOTCTRL_MULTI_OS := yes

```
e.g.,
meta/meta-mediatek-mt8668-hyp/conf/machine/auto8668p1_64_sos.conf
SUPPORT_BOOTCTRL_MULTI_OS := yes
meta/meta-mediatek-mt8668-hyp /conf/machine/auto8668p1_64_uos.conf
SUPPORT_BOOTCTRL_MULTI_OS := yes
```

注：请不要在单个 project.mk 或 project.conf 内同时支持不同类型的 OTA 升级方案

2.2 方案概述

2.2.1 基础功能

Hypervisor (L+L+A) 整体集中升级方案支持如下功能：

- 支持在设备后台无缝更新
- 支持系统回滚，避免升级失败后系统变砖
- 支持对升级包的完整性与准确性校验
- 支持一次升级多个 OS 系统
- 支持全包升级和差分升级
- 支持手动切换 A/B (需要执行至少一次 OTA 升级保证 A/B slot 均有可用的镜像信息)

- 支持断电接力升级

但它同时也具备如下限制：

- 不支持更新分区布局，即升级前后版本的分区布局必须保持一致，后续无法通过 OTA 修改分区布局
- 不支持 AB 分区 & non-AB 分区之间相互升级
- 需要升级的分区镜像必须是 xxx.img 格式，例如：boot.img，不支持其它格式的文件
- 需要升级的分区类型必须为只读 (RO) 分区
- 不支持 Android Virtual AB (安卓虚拟 AB)
- 不支持 OS 单独升级

2.2.2 Hypervisor OTA 名词解释

表 2-1. 名词解释

缩略词	全称和释义
Bootctl	Boot Control, A/B slot 标识结构体，存储在 misc 分区中
LK2	Little Kernel 2, 微型内核
OTA	Over-The-Air, 空中下载升级，一种远程更新技术
SOS	Server OS, Multi OS 系统内的 Host OS
UOS	User OS, Multi OS 系统内的 Guest OS

2.3 架构/进程概述

2.3.1 Hypervisor (L+L+A) OTA 升级架构

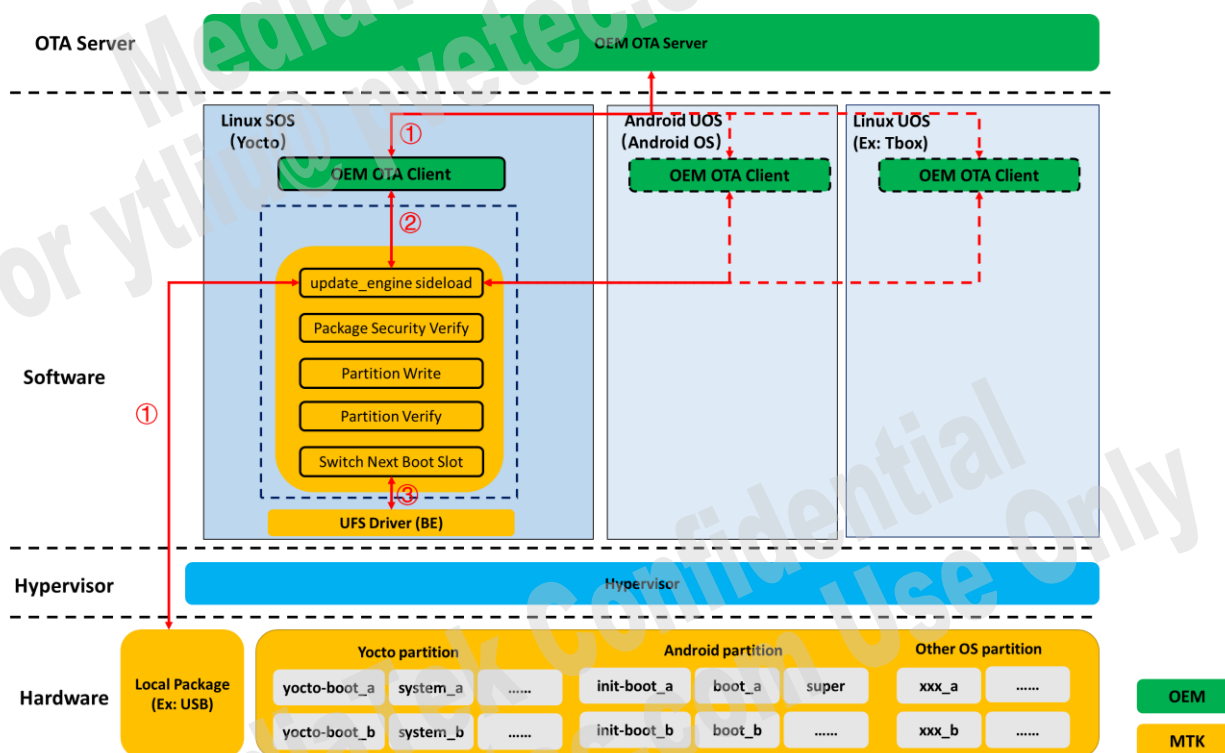


图 2-1. Hypervisor (L+L+A) OTA Update 架构

图 2-1 描述了 Hypervisor (L+L+A) 整体集中升级的方案架构，从图中能获取到如下信息：

1. Hypervisor (L+L+A) OTA 的升级包支持从 OTA Server 或者本地存储空间获取。MediaTek 目前仅支持通过本地升级包进行 OTA 升级，从 OTA Server 端利用网络下载升级包需要 OEM 自行实现。
2. Hypervisor OTA 升级全部在 Linux SOS (Yocto) 上完成，其它 UOS (Android UOS + Linux UOS) 不参与升级操作。
3. 以通过本地升级包完成 OTA 升级为例，Hypervisor (L+L+A) OTA 升级的大致流程如下：
 - 1) 将升级包存放到本地平台存储环境内；
 - 2) 执行 OTA update script 调用 update_engine sideload 触发升级；
 - 3) 利用 Package Security Verify 对升级包进行完整性与准确性校验；
 - 4) 利用 Partition Write 将升级包内的数据通过 UFS Driver 写入到对应分区存储空间内；
 - 5) 利用 Partition Verify 对更新写入到分区内的数据进行准确性校验，确保实际写入数据的准确性；
 - 6) 利用 Switch Next Boot Slot 设定下一次系统的启动插槽，确保系统下一次从更新后的插槽启动。

2.3.2 Hypervisor (L+L+A) 升级包编译架构

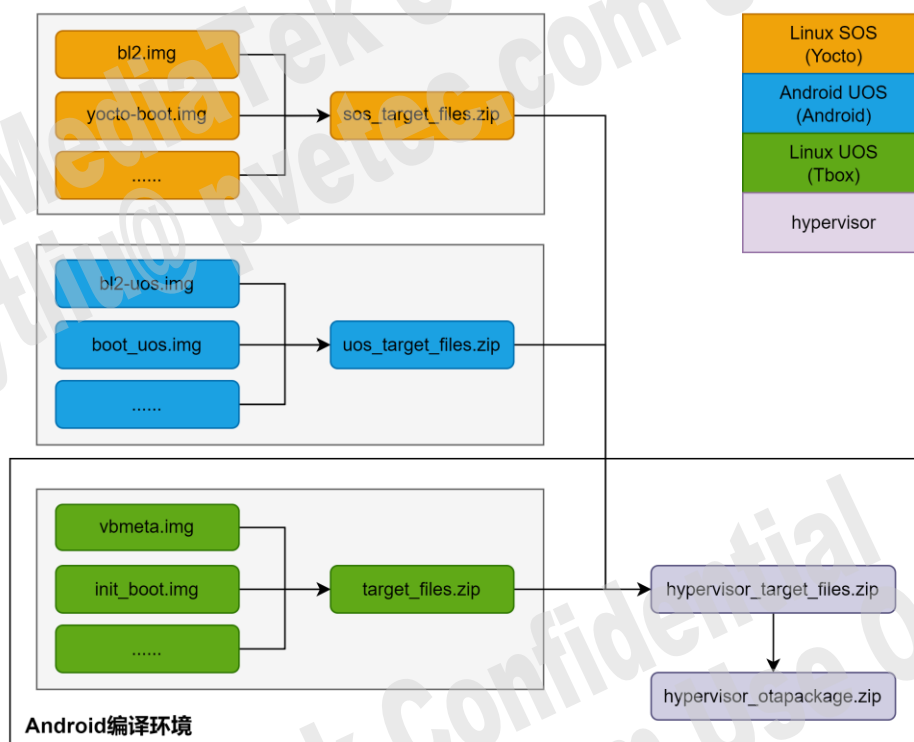


图 2-2. Hypervisor (L+L+A) build otapackage 架构

图 2-2 描述了 Hypervisor (L+L+A) OTA 整体集中升级方案编译升级包的架构，从图中能获取到如下信息：

1. Hypervisor (L+L+A) 整体集中升级只需要编译一个 OTA 升级包；
2. Hypervisor (L+L+A) 编译升级包必须在 Hypervisor Android 的编译环境下进行；
3. Hypervisor 编译升级包的大致流程如下：
 - 编译 Hypervisor sos_target_files.zip
 - 编译 Hypervisor uos_target_files.zip
 - 编译 Hypervisor android target_files.zip
 - 将 sos_target_files.zip、uos_target_files.zip 和 target_files.zip 合并成最终的 hypervisor_target_files.zip
 - 利用在 hypervisor_target_files.zip 在 Android 编译环境下编译生成全量升级包和差分升级包

注：关于编译 Hypervisor OTA 升级包的详细说明，请参考章节 2.4 如何编译 Hypervisor (L+L+A) OTA 升级包

2.4 如何编译 Hypervisor (L+L+A) OTA 升级包

2.4.1 编译 hypervisor_target_files.zip

1. Build sos_target_files.zip

Linux SOS (Yocto) 的 target_files.zip 由 hypervisor_make_sos_targetfiles.py 编译产生，脚本位于：

```
meta/meta-mediatek-mt8668-hyp/recipes-devtools/ota-tools/files
```

在 Linux SOS (Yocto)编译阶段，系统会自动执行该脚本进行打包，最终的 sos_target_files.zip 位于：

```
build/tmp/deploy/images/<project_name>/sos_target_files.zip
```

2. Build uos_target_files.zip

Linux UOS (Tbox)的 target_files.zip 由 hypervisor_make_uos_targetfiles.py 编译产生，脚本位于：

```
meta/meta-mediatek-mt8668-hyp/recipes-devtools/ota-tools /files
```

在 Linux UOS (Tbox)编译阶段，系统会自动执行该脚本进行打包，最终的 uos_target_files.zip 位于：

```
build/tmp/deploy/images/<project_name>/uos_target_files.zip
```

3. Build android target_files.zip

Android 的 target_files.zip 在 build Android OS 阶段自动编译产生，最终的 target_files.zip 位于：

```
out/target/product/<project name>/merged/target_files.zip
```

4. Build hypervisor_target_files.zip

hypervisor_target_files.zip 需要使用 hypervisor_multi_os_merge_targetfiles.py 脚本去合并 sos_target_files.zip、uos_target_files.zip 和 target_files.zip 后产生，hypervisor_multi_os_merge_targetfiles.py 位于：

```
vendor/mediatek/proprietary/scripts/hypervisor_ota_tool/hypervisor_multi_os_merge_targetfiles.py
```

在获取到 sos_target_files.zip、uos_target_files.zip 和 target_files.zip 后，在 Android 的编译环境下执行如下指令去获取 hypervisor_target_files.zip：

```
python3 hypervisor_multi_os_merge_targetfiles.py <参数 1> <参数 2> <参数 3>
```

注：

- hypervisor_multi_os_merge_targetfiles.py：编译 hypervisor_target_files.zip 的脚本
- <参数 1>：sos_target_files.zip 的路径
- <参数 2>：uos_target_files.zip 的路径
- <参数 3>：target_files.zip 的路径

输出文件：

在当前目录下会产生名称为 hypervisor_target_files.zip 的文件，这就是 Hypervisor(L+L+A)的 target_files.zip 包

示例：

```
python3
vendor/mediatek/proprietary/scripts/hypervisor_ota_tool/hypervisor_multi_os_merge_targetfile
s.py 3OS_target_files/sos_target_files.zip 3OS_target_files/uos_target_files.zip
3OS_target_files/target_files.zip
```

2.4.2 编译 Hypervisor (L+L+A) 全量升级包

在参考 2.4.1 编译 `hypervisor_target_files.zip` 小节编译得到 `hypervisor_target_files.zip` 后，就可以在 Hypervisor (L+L+A) Android OS 的编译环境下去编译 Hypervisor(L+L+A)的全量升级包。

编译指令：

```
./out_sys/host/linux-x86/bin/ota_from_target_files -v --block -p out_sys/host/linux-x86 -k
build/make/target/product/security/testkey hypervisor_target_files.zip otapackage_full.zip
```

注：

- `ota_from_target_files`：升级包编译脚本
- `build/make/target/product/security/testkey`：对升级包进行签名的 key 路径，Mediatek 默认使用 Google 提供的 testkey，贵司可指定其它的 key 路径，使用 -k 参数指定。如何更换升级包签名 key，请参考：章节 2.7 如何修改升级包的签名
- `hypervisor_target_files.zip`：Hypervisor (L+L+A)的 `target_files.zip`
- `otapackage_full.zip`：最终产生的 Hypervisor 全量升级包，包名可随意指定
- 请务必确保在 Hypervisor (L+L+A) Android OS 的编译环境下编译升级包，并确保编译全量升级包前有执行 source & lunch，否则将导致全量编译失败

2.4.3 编译 Hypervisor (L+L+A) 差分升级包

前提条件：

编译差分升级包需要预先准备两份 `hypervisor_target_files.zip`。一份是基底版本（source 版本）的 `hypervisor_target_files.zip`，此处命名为 `source_hypervisor_target_files.zip`；一份是目标版本（target 版本）的 `hypervisor_target_files.zip`，此处命名为 `target_hypervisor_target_files.zip`。

注：请务必确保平台上烧录的基底版本 load 和基底版本的 `hypervisor_target_files.zip` 是同一次编译得到的，否则将导致 OTA 差分升级失败。

编译指令：

```
./out_sys/host/linux-x86/bin/ota_from_target_files -v --block -p out_sys/host/linux-x86 -k
build/make/target/product/security/testkey -i source_hypervisor_target_files.zip
target_hypervisor_target_files.zip otapackage_delta.zip
```

注:

- ota_from_target_files: 升级包编译脚本
- build/make/target/product/security/testkey: 对升级包进行签名的 key 路径, MediaTek 默认使用 Google 提供的 testkey, 贵司可指定其它的 key 路径, 使用-k 参数指定。如何更换升级包签名 key, 请参考: 章节 2.7 如何修改升级包的签名
- source_hypervisor_target_files.zip: source 版本的 hypervisor_target_files.zip
- target_hypervisor_target_files.zip: target 版本的 hypervisor_target_files.zip
- otapackage_delta.zip: 最终产生的 Hypervisor (L+L+A)差分升级包, 包名可随意指定
- 请务必确保在 Hypervisor Android 的 build 环境下编译升级包, 并确保编译全包前有执行 source & lunch, 否则将导致差分包编译失败

2.5 升级包结构解析

Hypervisor (L+L+A) OTA 升级包是一个被压缩的 zip 包, 其主要组成部分如下:

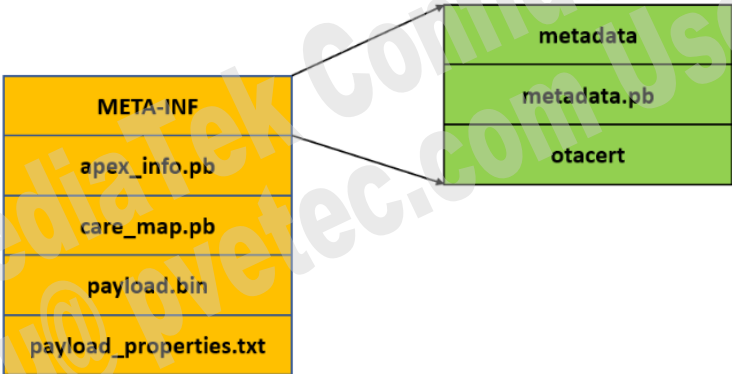


图 2-3. Hypervisor (L+L+A) otapackage 结构

图 2-3 中 Hypervisor (L+L+A) otapackage 的各个组成部分描述如下表所示:

表 2-2. 升级包组件描述

名称	描述
metadata	存储升级包的 metadata 信息, 包括升级版本、升级类型等
metadata.pb	存储升级包的 metadata 信息, 包括升级版本、升级类型等
otacert	升级包进行签名验证信息
apex_info.pb	Android apex 模块配置信息
care_map.pb	系统 fingerprint 信息
payload.bin	升级镜像信息

名称	描述
payload_properties.txt	payload.bin 和 metadata 的 size 和 hash 信息

从表 2-2 内可以看出，系统的升级信息都存储在 payload.bin 内，其组成结构如下：

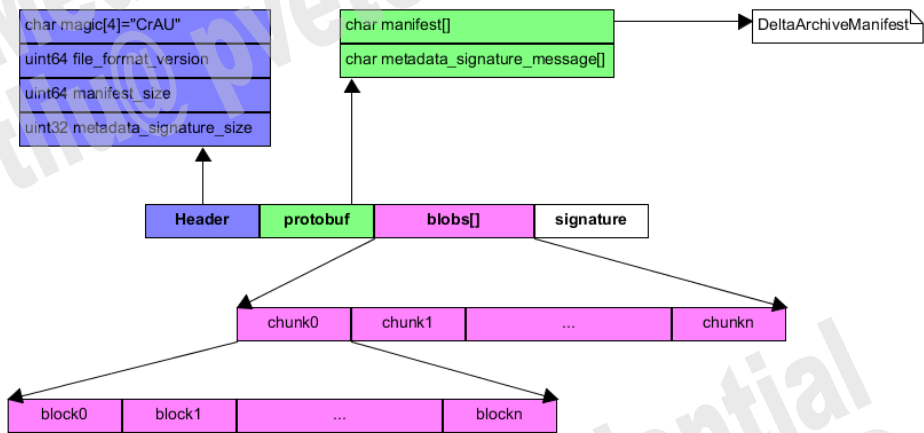


图 2-4. payload.bin 组成结构

从图 2-4 中可以看出，payload.bin 的主要组成部分如表 2-3 所示：

表 2-3. payload.bin 组件描述

名称	名称	描述
Header	Magic Number	Magic string "CrAU" identifying this is an update payload.
	Major Version	Payload major version number.
	Manifest Size	Manifest size in bytes.
	Manifest Signature Size	Manifest signature blob size in bytes
protobuf	Manifest	The list of operations to be performed.
protobuf	Manifest Signature	The signature of the first five fields. There could be multiple signatures if the key has changed.
blobs []	Payload Data	The list of binary blobs used by operations in the metadata.
signature	Payload Signature	The signature of the entire payload except the metadata signature. There could be multiple signatures if the key has changed.

2.6 如何进行 Hypervisor (L+L+A) OTA 升级

本地验证 OTA 升级的脚本位于 Linux SOS (Yocto) 代码的如下路径：

```
src/bsp/lk2/platform/mediatek/common/bootctrl/scripts/hypervisor_update.py
```

2.6.1 全包升级 (Normal Mode)

测试环境

1. The PC has Python3 environment
2. Flash base load in DUT
3. Get OTA package

测试步骤

1. Device connection yocto adb environment
2. Execute the upgrade script: python3 `hypervisor_update.py` --file `otapackage_full.zip` > `update.txt 2>&1`
 - `hypervisor_update.py`: Update script
 - `otapackage_full.zip`: OTA package, unlimited package name
 - > `update.txt 2>&1`: Store the upgrade log in update.txt in the current directory

预期结果

1. Update should be completed, and `update.txt` prints the following log: [INFO:update_attempter_android.cc(600)]
Update successfully applied, waiting to reboot.
2. The first restart after the upgrade should successfully enter the HomeScreen
3. Check whether setting-version has been updated
Note: If the test upgrade fails, please provide the update log and uart log in CR.

2.6.2 差分升级 (Normal Mode)

测试环境

1. The PC has Python3 environment
2. Flash base load in DUT, **must be consistent with the base version provided to the OTA owner**
3. Get OTA package from OTA owner

测试步骤

1. Device connection yocto adb environment
2. Execute the upgrade script: python3 `hypervisor_update.py` --file `otapackage_delta.zip` > `update.txt 2>&1`
 - `hypervisor_update.py`: Update script
 - `otapackage_delta.zip`: OTA package, unlimited package name
 - > `update.txt 2>&1`: Store the upgrade log in update.txt in the current directory

预期结果

1. Update should be completed, and `update.txt` prints the following log: [INFO:update_attempter_android.cc(600)]
Update successfully applied, waiting to reboot.
2. The first restart after the upgrade should successfully enter the HomeScreen
3. Check whether setting-version has been updated
Note: If the test upgrade fails, please provide the update log and uart log in CR.

2.7 如何修改升级包的签名

OTA 升级过程中分别会使用到两把 key：xxx.pk8 和 xxx.x509.pem，其中 xxx.pk8 格式的 key 用于在编译升级包时对升级包进行签名，xxx.x509.pem 格式的 key 则用于在 OTA 升级过程中对升级包进行校验。

如果您需要使用定制化的 key，请按照如下步骤操作：

1. 将章节 0
2. 编译 Hypervisor (L+L+A) 全量升级包和 2.4.3 编译 Hypervisor (L+L+A) 差分升级包中升级包编译指令 -k 参数的指定路径替换为实际使用的 xxx.pk8 和 xxx.x509.pem 路径；
3. 将 meta/meta-mediatek/recipes-support/update-engine-sideload-u/files/ota/ otacerts.zip 内更换实际使用的 xxx.x509.pem 文件。

2.8 如何删减/新增 OTA 升级分区

2.8.1 OTA 升级分区的来源

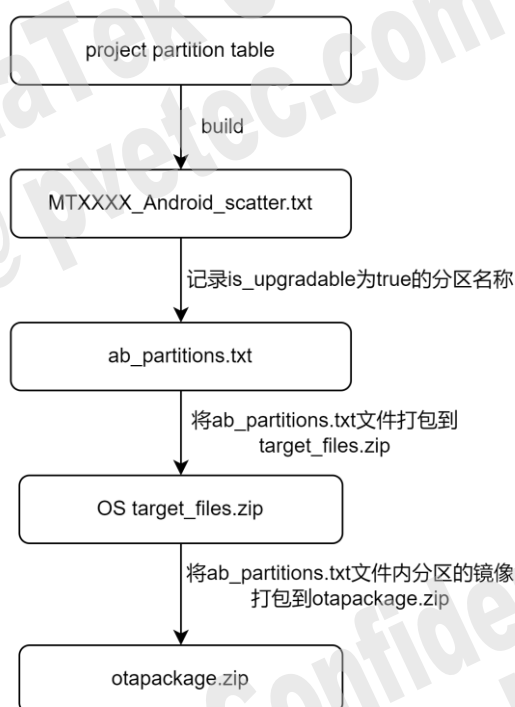


图 2-5. OTA 升级分区来源

OTA 升级分区的确认步骤如下：

1. 编译阶段将分区表内的信息打包转化为 MTXXXX_Android_scatter.txt，Ex: MT6897_Android_scatter.txt；
2. 将 MTXXXX_Android_scatter.txt 内 is_upgradable 为 true 的分区记录到 ab_partitions.txt；

3. 将 ab_partitions.txt 打包到各个 OS 的 target_files.zip;
4. 编译升级包时将 ab_partitions.txt 内记录的分区镜像打包到升级包内。

注：检查分区 is_upgradable 状态时会优先判断_a 分区的 is_upgradable，例如：当 xxxx_a 分区的 is_upgradable 为 true，xxx_b 的 is_upgradable 为 false 时，系统仍会记录该分区为 OTA 升级分区。

2.8.2 删减 OTA 升级分区

删减 OTA 升级分区只会关闭特定 AB 分区的 OTA 升级，并不会将关闭 AB 升级的分区修改为单分区。

Hypervisor (L+L+A)包含三个不同的 OS project，其镜像组成包括：Linux SOS (Yocto)分区镜像+ Android UOS (Android) 分区镜像 + Linux UOS (Tbox)分区镜像。Hypervisor (L+L+A)也包含两份分区表，一份分区表位于 Linux SOS (Yocto) codebase 内，分区表内包含 Yocto + Android + Tbox 的所有分区信息，该分区表在编译 Yocto 和 Tbox 阶段被使用；另一份分区表位于 Android codebase 内，分区表内仅包含 Android 分区信息，该分区表仅在编译 Android 阶段被使用。

Yocto & Tbox 分区表：

src/bsp/ptgen_v2/mt6881/partition_table_emmc_hyp_ab.csv

Android 分区表：

device/mediateksample/<project_name>/partition_table_emmc_hyp_ab.csv

如下为各个 OS 删减 OTA 升级分区的方法：

1. Linux SOS (Yocto)

将 Yocto 分区表内需要关闭 OTA 升级分区一栏的 OTA_Update 状态从 Y 改成 N

e.x.,

修改前：

```
preloader_a,Raw data,1024,EMMC_BOOT1,UFS_LU0,N,Y,bl2.img,Y,N,BOOTLOADERS,Y
preloader_b,Raw data,1024,EMMC_BOOT2,UFS_LU1,N,Y,bl2.img,Y,N,BOOTLOADERS,Y
```

修改后：

```
preloader_a,Raw data,1024,EMMC_BOOT1,UFS_LU0,N,Y,bl2.img,N,N,BOOTLOADERS,Y
preloader_b,Raw data,1024,EMMC_BOOT2,UFS_LU1,N,Y,bl2.img,N,N,BOOTLOADERS,Y
```

2. Android UOS (Android)

将 Yocto 分区表内需要关闭 OTA 升级分区一栏的 OTA_Update 状态从 Y 改成 N，Android 分区表无需修改

e.x.,

修改前:

init_boot_a,Raw data,8192,EMMC_USER,UFS_LU2,N,Y,init_boot.img,**Y**,N,AUTO,Y

init_boot_b,Raw data,8192,EMMC_USER,UFS_LU2,N,N,NONE,**N**,N,AUTO,Y

修改后:

init_boot_a,Raw data,8192,EMMC_USER,UFS_LU2,N,Y,init_boot.img,**N**,N,AUTO,Y

init_boot_b,Raw data,8192,EMMC_USER,UFS_LU2,N,N,NONE,**N**,N,AUTO,Y

3. Linux UOS (Tbox)

将 Tbox 分区表内需要关闭 OTA 升级分区一栏的 OTA_Update 状态从 Y 改成 N

e.g.,

修改前:

bl2-uos_a,Raw data,1024,EMMC_USER,UFS_LU2,N,Y,bl2-uos.img,**Y**,N,AUTO,Y

bl2-uos_b,Raw data,1024,EMMC_USER,UFS_LU2,N,N,NONE,**Y**,N,AUTO,Y

修改后:

bl2-uos_a,Raw data,1024,EMMC_USER,UFS_LU2,N,Y,bl2-uos.img,**N**,N,AUTO,Y

bl2-uos_b,Raw data,1024,EMMC_USER,UFS_LU2,N,N,NONE,**N**,N,AUTO,Y

2.8.3 新增 OTA 升级分区

新增的 OTA 升级分区必须满足如下前提条件:

1. 分区必须同时包含<partition_name>_a & <partition_name>_b, 例如: boot_a & boot_b
2. 分区必须为只读 (RO) 分区
3. 分区镜像格式和镜像名称必须为<partition_name>.img, 例如: boot.img

2.8.3.1 设定已存在的 AB 分区为 OTA 分区

各个 OS 将分区表内已经存在的 AB 分区设定为 OTA 分区的步骤如下:

1. Linux SOS (Yocto)

将 Yocto 分区表内需要打开 OTA 升级分区一栏的 OTA_Update 状态从 N 改成 Y

e.x.,

修改前:

preloader_a,Raw data,1024,EMMC_BOOT1,UFS_LU0,N,Y,bl2.img,**N**,N,BOOTLOADERS,Y

preloader_b,Raw data,1024,EMMC_BOOT2,UFS_LU1,N,Y,bl2.img,**N**,N,BOOTLOADERS,Y

修改后:

preloader_a,Raw data,1024,EMMC_BOOT1,UFS_LU0,N,Y,bl2.img,**Y**,N,BOOTLOADERS,Y

preloader_b,Raw data,1024,EMMC_BOOT2,UFS_LU1,N,Y,bl2.img,**Y**,N,BOOTLOADERS,Y

2. Android UOS (Android)

将 Yocto 分区表 & Android 分区表内需要打开 OTA 升级分区一栏的 OTA_Update 状态从 N 改成 Y

e.g.,

修改前:

```
init_boot_a,Raw data,8192,EMMC_USER,UFS_LU2,N,Y,init_boot.img,N,N,AUTO,Y
```

```
init_boot_b,Raw data,8192,EMMC_USER,UFS_LU2,N,N,NONE,N,N,AUTO,Y
```

修改后:

```
init_boot_a,Raw data,8192,EMMC_USER,UFS_LU2,N,Y,init_boot.img,Y,N,AUTO,Y
```

```
init_boot_b,Raw data,8192,EMMC_USER,UFS_LU2,N,N,NONE,Y,N,AUTO,Y
```

在 hypervisor_make_sos_targetfiles.py 的 ignored_partitions 内添加新增的 Android OTA 分区名, 用于编译 sos_target_files.zip 时跳过该分区, 避免编译阶段报错分区镜像不存在的错误

e.g.,

```
ignored_partitions = {'dtbo', 'boot', 'vendor_boot', 'init_boot', 'userdata', 'super', 'vbmeta', 'vbmeta_system',  
'vbmeta_vendor', 'tee', 'bl2-uos', 'boot_uos'} >> 将新增的分区添加到列表
```

3. Linux UOS (Tbox)

将 Tbox 分区表内需要关闭 OTA 升级分区一栏的 OTA_Update 状态从 N 改成 Y

e.g.,

修改前:

```
bl2-uos_a,Raw data,1024,EMMC_USER,UFS_LU2,N,Y,bl2-uos.img,N,N,AUTO,Y
```

```
bl2-uos_b,Raw data,1024,EMMC_USER,UFS_LU2,N,N,NONE,N,N,AUTO,Y
```

修改后:

```
bl2-uos_a,Raw data,1024,EMMC_USER,UFS_LU2,N,Y,bl2-uos.img,Y,N,AUTO,Y
```

```
bl2-uos_b,Raw data,1024,EMMC_USER,UFS_LU2,N,N,NONE,Y,N,AUTO,Y
```

在 hypervisor_make_sos_targetfiles.py 的 ignored_partitions 内添加新增的 Tbox OTA 分区名, 用于编译 sos_target_files.zip 时跳过该分区, 避免编译阶段报错分区镜像不存在的错误

e.g.,

```
ignored_partitions = {'dtbo', 'boot', 'vendor_boot', 'init_boot', 'userdata', 'super', 'vbmeta', 'vbmeta_system',  
'vbmeta_vendor', 'tee', 'bl2-uos', 'boot_uos'} >> 将新增的分区添加到列表
```

在 hypervisor_make_uos_targetfiles.py 的 partitions 内添加新增的 Tbox OTA 分区名, 确保 Tbox 编译阶段将分区镜像打包到 uos_target_files.zip

e.g.,

```
if __name__ == "__main__":
```

```
    target_path = "uos_images"
```

```
    partitions = ['bl2-uos', 'boot_uos'] >> 将新增的分区添加到列表
```

2.8.3.2 新增不存在的 AB 分区为 OTA 分区

各个 OS 新增分区表内不存在的 AB 分区为 OTA 分区的步骤如下：

1. Linux SOS (Yocto)

在 Yocto 分区表内新增对应 AB 分区，并确保分区镜像为<partition_name>.img，以新增 customer 分区为例：

修改后(分区大小设定为 8MB)：

```
customer_a,Raw data,8192,EMMC_USER,UFS_LU2,N,Y,customer.img,Y,N,AUTO,Y
customer_b,Raw data,8192,EMMC_USER,UFS_LU2,N,N,NONE,Y,N,AUTO,Y
```

分区添加完毕后，请在 customer 分区的启动加载阶段添加 AB 分区逻辑判断，保证系统加载到正确的分区

2. Android UOS (Android)

在 Yocto 分区表和 Android 分区表内新增对应 AB 分区，并确保分区镜像为<partition_name>.img，以新增 customer 分区为例：

修改后(分区大小设定为 8MB)：

```
customer_a,Raw data,8192,EMMC_USER,UFS_LU2,N,Y,customer.img,Y,N,AUTO,Y
customer_b,Raw data,8192,EMMC_USER,UFS_LU2,N,N,NONE,Y,N,AUTO,Y
```

在 hypervisor_make_sos_targetfiles.py 的 ignored_partitions 内添加新增的 Android OTA 分区名，用于编译 sos_target_files.zip 时跳过该分区，避免编译阶段报错分区镜像不存在的错误

e.g.,
ignored_partitions = {'dtbo', 'boot', 'vendor_boot', 'init_boot', 'userdata', 'super', 'vbmeta', 'vbmeta_system', 'vbmeta_vendor', 'customer', 'tee', 'bl2-uos', 'boot_uos'} >> 将新增的分区添加到列表

3. Linux UOS (Tbox)

在 Yocto 分区表内新增对应 AB 分区，并确保分区镜像为<partition_name>.img，以新增 customer 分区为例：

修改后(分区大小设定为 8MB)：

```
customer_a,Raw data,8192,EMMC_USER,UFS_LU2,N,Y,customer.img,Y,N,AUTO,Y
customer_b,Raw data,8192,EMMC_USER,UFS_LU2,N,N,NONE,Y,N,AUTO,Y
```

在 hypervisor_make_sos_targetfiles.py 的 ignored_partitions 内添加新增的 Tbox OTA 分区名，用于编译 sos_target_files.zip 时跳过该分区，避免编译阶段报错分区镜像不存在的错误

e.g.,
ignored_partitions = {'dtbo', 'boot', 'vendor_boot', 'init_boot', 'userdata', 'super', 'vbmeta', 'vbmeta_system', 'vbmeta_vendor', 'tee', 'bl2-uos', 'boot_uos', 'customer'} >> 将新增的分区添加到列表

在 hypervisor_make_uos_targetfiles.py 的 partitions 内添加新增的 Tbox OTA 分区名，确保 Tbox 编译阶段将分区镜像打包到 uos_target_files.zip

```
e.g.,:
if __name__ == "__main__":
    target_path = "uos_images"
    partitions = ['bl2-uos', 'boot_uos', 'customer'] >>将新增的分区添加到列表
```

- 注:
- 1. 新增不存在的 AB 分区为 OTA 分区时, 请确保新增分区挂载成只读分区
 - 2. 新增不存在的 AB 分区为 OTA 分区时, 请确保新增分区后系统烧录能正常加载启动, 然后再验证 OTA 升级

2.9 AB 系统的镜像加载选择

2.9.1 Boot Control 信息说明

AB 系统包含两个启动 slot: slot_a 和 slot_b, 每个 slot 有自己独立的一份 Boot Control metadata。系统启动会根据存储在 misc 分区的 Boot Control 信息来选择加载 slot_a 或 slot_b 的分区镜像, 其中 Boot control 包含的关键信息如下:

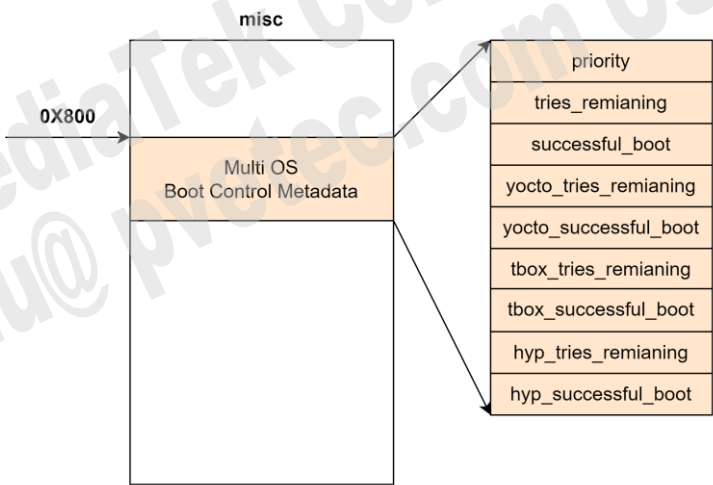


图 2-6. Hypervisor (L+L+A) Boot Control 参数

图 2-6 中各个 boot control 参数的说明如表 2-4 所示:

表 2-4. Hypervisor (L+L+A) Boot control 参数说明

名称	描述
priority	Slot 的启动优先级, 初始值默认为 15, 系统选择从优先级高的 slot 启动, 优先级相同则从 slot_a 启动

名称	描述
tries_remaining	当前 slot android 的剩余可重启次数，初始值默认为 7，值为 0 时会触发系统回滚
successful_boot	android 启动成功标志位，1：启动成功，0：启动失败
yocto_tries_remaining	当前 slot yocto 的剩余可重启次数，初始值默认为 3，值为 0 时会触发系统回滚
yocto_successful_boot	yocto 启动成功标志位，1：启动成功，0：启动失败
tbox_tries_remaining	当前 slot tbox 的剩余可重启次数，初始值默认为 3，值为 0 时会触发系统回滚
tbox_successful_boot	tbox 启动成功标志位，1：启动成功，0：启动失败
hyp_tries_remaining	当前 slot hypervisor 的剩余可重启次数，初始值默认为 3，值为 0 时会触发系统回滚
hyp_successful_boot	hypervisor 启动成功标志位，1：启动成功，0：启动失败

注：Boot Control Source Code 位于：src/bsp/lk2/platform/mediatek/common/bootctrl/bootctrl_multi_os

2.9.2 镜像加载选择流程

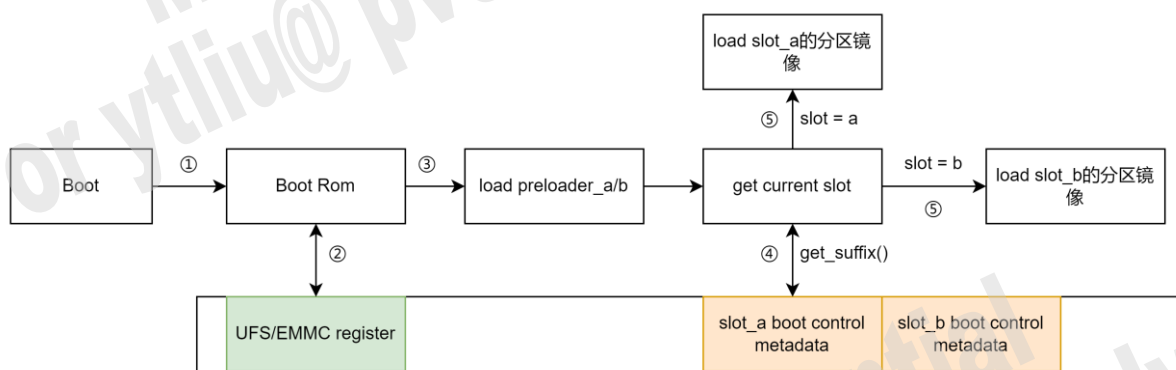


图 2-7. Hypervisor (L+L+A) 镜像加载流程

AB 系统启动的镜像加载流程如图 2-7 所示：

- ① 系统启动进入 Boot Rom 阶段；
- ② Boot Rom 会读取 UFS/EMMC 上特定寄存器的值来决定 load preloader_a 或 preloader_b 分区镜像；
- ③ 系统启动进入 preloader 阶段
- ④ 系统去读取 Boot Control 信息内的 slot_a 与 slot_b 的 priority 值

- ⑤ 当 slot_a 的 priority 值大于等于 slot_b 的 priority 值时，系统判断当前启动 slot 为 slot_a，后续启动过程中会加载 A 分区内的镜像；当 slot_a 的 priority 值小于 slot_b 的 priority 值时，系统判断当前启动 slot 为 slot_b，后续启动过程中会加载 B 分区内的镜像。

注：preloader 分区的 AB 加载选择由 UFS/EMMC 上特定寄存器的值来决定，不由 boot control metadata 的 slot priority 决定，详细信息请咨询 Boot Rom & preloader owner

2.10 系统回滚

本节主要介绍 Hypervisor(L+L+A) OTA 升级完成后，当系统首次从新版本启动失败时，系统如何回滚到升级前的旧版本启动，避免设备变砖。

2.10.1 系统回滚的基本原则

由于 Hypervisor (L+L+A) Project 涉及到多个 OS，因此 MediaTek 采用的整体回滚方案遵循如下原则：

- Hypervisor OS & Linux OS(Yocto + Tbox) & Android OS 整体 rollback，即当有任意一个 OS 首次从新版本启动失败时，所有 OS 都要回滚到旧系统启动，保证所有 OS 的 AB 启动 slot 一致
- 只有在 OTA 升级成功后首次从新系统启动失败的场景下才会触发回滚机制，常规启动失败不会触发回滚机制

以 Hypervisor、Yocto、Tbox 和 Android 的 3OS (L+L+A) OTA 为例，系统会存在如下四种 rollback 场景：

场景一：Hypervisor OS 从新版本启动失败

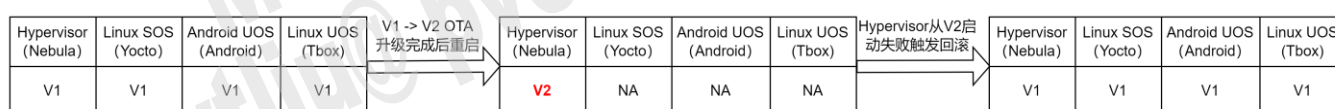


图 2-8. OTA 升级后 Hypervisor 启动失败场景

图 2-8 描述 Hypervisor (L+L+A) OTA 升级后，当 Hypervisor 从新版本启动失败触发 rollback 时，Hypervisor、Yocto、Android 的版本切换状态，场景描述如下：

- Hypervisor & Yocto & Android & Tbox 都从 V1 版本启动
- 系统执行 OTA 升级到 V2 版本，并将 V2 版本设定为下一次启动的版本
- Hypervisor 从 V2 版本启动失败，此时 Yocto、Android 和 Tbox 都还未启动
- 触发回滚，Hypervisor & Yocto & Android & Tbox 都切换到 V1 版本启动

场景二：Yocto OS 从新版本启动失败**图 2-9. OTA 升级后 Yocto 启动失败场景**

图 2-9 描述 Hypervisor (L+L+A) OTA 升级后，当 Hypervisor 从新版本 V2 启动成功，Yocto 从新版本启动失败触发 rollback 时，Hypervisor、Yocto、Android 和 Tbox 的版本切换状态，场景描述如下：

- Hypervisor & Yocto & Android & Tbox 都从 V1 版本启动
- 系统执行 OTA 升级到 V2 版本，并将 V2 版本设定为下一次启动的版本
- Hypervisor 从 V2 版本启动成功，Yocto 从 V2 版本启动失败，此时 Android 和 Tbox 的启动状态未知
- 触发回滚，Hypervisor & Yocto & Android & Tbox 都切换到 V1 版本启动

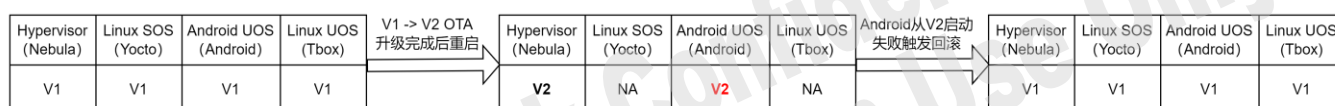
场景三：Android OS 从新版本启动失败**图 2-10. OTA 升级后 Android 启动失败场景**

图 2-10 描述了 Hypervisor(L+L+A) 从 V1 -> V2 OTA 升级后，当 Hypervisor 从新版本 V2 启动成功，Android 从新版本 V2 启动失败触发 rollback 时，Hypervisor、Yocto、Android 和 Tbox 的版本切换状态，场景描述如下：

- Hypervisor & Yocto & Android & Tbox 都从 V1 版本启动
- 系统执行 OTA 升级到 V2 版本，并将 V2 版本设定为下一次启动的版本
- Hypervisor 从 V2 版本启动成功，Android 从 V2 版本启动失败，此时 Yocto 和 Tbox 的启动状态未知
- 触发回滚，Hypervisor & Yocto & Android & Tbox 都切换到 V1 版本启动

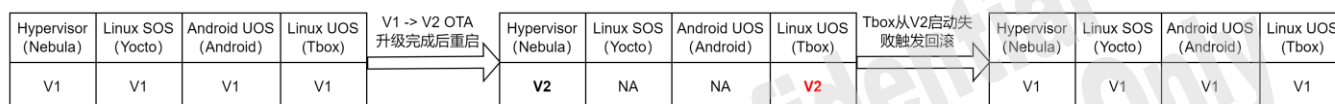
场景四：Tbox OS 从新版本启动失败**图 2-11. OTA 升级后 Tbox 启动失败场景**

图 2-11 描述了 Hypervisor(L+L+A) 从 V1 -> V2 OTA 升级后，当 Hypervisor 从新版本 V2 启动成功，Tbox 从新版本 V2 启动失败触发 rollback 时，Hypervisor、Yocto、Android 和 Tbox 的版本切换状态，场景描述如下：

- Hypervisor & Yocto & Android & Tbox 都从 V1 版本启动

- 系统执行 OTA 升级到 V2 版本，并将 V2 版本设定为下一次启动的版本
- Hypervisor 从 V2 版本启动成功，Tbox 从 V2 版本启动失败，此时 Yocto 和 Android 的启动状态未知
- 触发回滚，Hypervisor & Yocto & Android & Tbox 都切换到 V1 版本启动

2.10.2 系统回滚的检测流程

Hypervisor (L+L+A) 的系统回滚检测流程设定在各个 OS 的 LK2 阶段，其详细流程如图 2-12 所示：

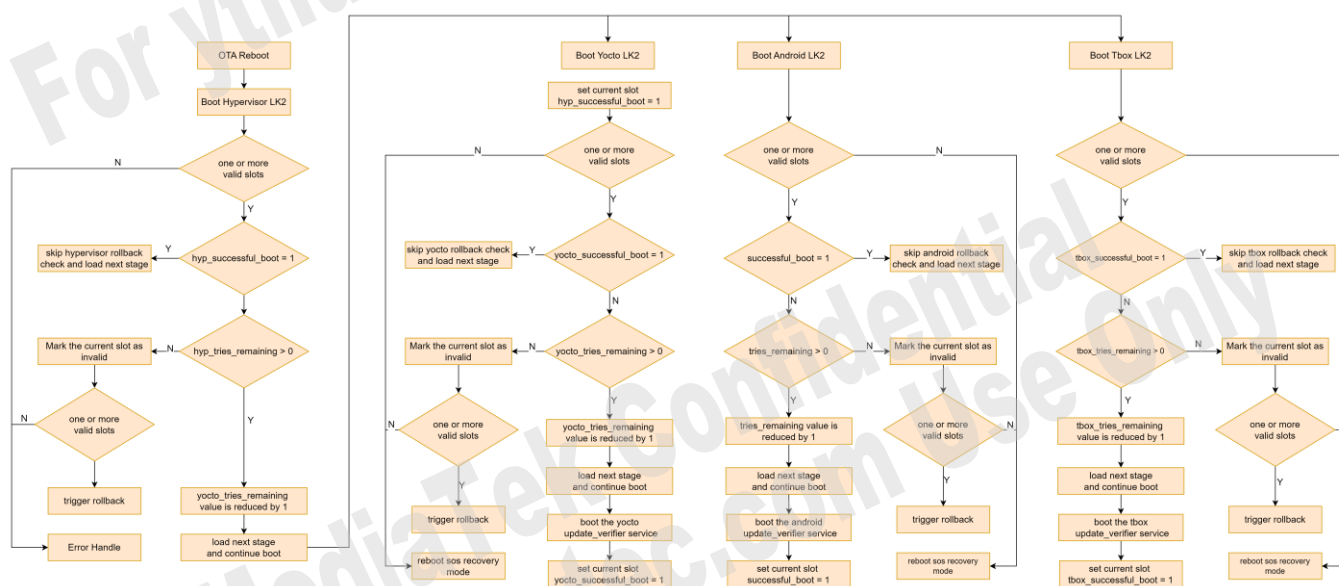


图 2-12. Hypervisor(L+L+A) 回滚检测流程

2.10.3 系统回滚的 Boot Control 参数变化

由图 2-12 可知，系统回滚机制的本质就是通过反复的判断和调整 Boot Control 信息，且只有触发系统的反复重启让某个系统的 tries_remaining 值减少到 0 才能触发系统整体回滚。Hypervisor(L+L+A) 3OS 的启动可分为如下五种场景：

场景一：Hypervisor & Yocto & Android & Tbox 都启动成功（OTA 升级后重启）

Hypervisor (Nebula)	Linux SOS (Yocto)	Android UOS (Android)	Linux UOS (Tbox)
OK	OK	OK	OK

图 2-13. Hypervisor & Yocto & Android & Tbox 启动状态

各个阶段的 Boot Control 参数值变化如下所示：

		Normal boot			No reboot after OTA upgrade			Hypervisor reboot successfully after OTA upgrade			Yocto reboot successfully after OTA upgrade			Android reboot successfully after OTA upgrade			Tbox reboot successfully after OTA upgrade	
		slot a	slot b		slot a	slot b		slot a	slot b		slot a	slot b		slot a	slot b		slot a	slot b
	priority	15	15		14	15		14	15		14	15		14	15		14	15
hypervisor	yocto_tries_remaining	3	3		1	3		1	1		1	1		1	1		1	1
	hyp_successful_boot	1	0		1	0		1	1		1	1		1	1		1	1
yocto	yocto_tries_remaining	3	3		1	3		1	3		1	1		1	1		1	1
	yocto_successful_boot	1	0		1	0		1	0		1	1		1	1		1	1
android	tries_remaining	7	7		1	7		1	7		1	7		1	1		1	1
	successful_boot	1	0		1	0		1	0		1	0		1	1		1	1
tbox	tbox_tries_remaining	3	3		1	3		1	3		1	3		1	3		1	1
	tbox_successful_boot	1	0		1	0		1	0		1	0		1	0		1	1

场景二：Hypervisor 启动失败，Yocto & Android & Tbox 均为启动（OTA 升级后重启）

Hypervisor (Nebula)	Linux SOS (Yocto)	Android UOS (Android)	Linux UOS (Tbox)
NG	NA	NA	NA

图 2-14. Hypervisor & Yocto & Android & Tbox 启动状态

各个阶段的 Boot Control 参数值变化如下所示：

		Normal boot			No reboot after OTA upgrade			Hypervisor reboot fail after OTA upgrade			Not boot			Not boot			Not boot			Triggering system rollback	
		slot a	slot b		slot a	slot b		slot a	slot b		slot a	slot b		slot a	slot b		slot a	slot b		slot a	slot b
	priority	15	15		14	15		14	15		14	15		14	15		14	15		14	0
hypervisor	yocto_tries_remaining	3	3		1	3		1	0		1	0		1	0		1	0		1	0
	hyp_successful_boot	1	0		1	0		1	0		1	0		1	0		1	0		1	0
yocto	yocto_tries_remaining	3	3		1	3		1	3		1	3		1	3		1	3		1	0
	yocto_successful_boot	1	0		1	0		1	0		1	0		1	0		1	0		1	0
android	tries_remaining	7	7		1	7		1	7		1	7		1	7		1	7		1	0
	successful_boot	1	0		1	0		1	0		1	0		1	0		1	0		1	0
tbox	tbox_tries_remaining	3	3		1	3		1	3		1	3		1	3		1	3		1	0
	tbox_successful_boot	1	0		1	0		1	0		1	0		1	0		1	0		1	0

场景三：Hypervisor & Tbox & Android 启动成功，Yocto 启动失败（OTA 升级后重启）

Hypervisor (Nebula)	Linux SOS (Yocto)	Android UOS (Android)	Linux UOS (Tbox)
OK	NG	OK	OK

图 2-15. Hypervisor & Yocto & Android & Tbox 启动状态

各个阶段的 Boot Control 参数值变化如下所示：

		Normal boot			No reboot after OTA upgrade			Hypervisor reboot successfully after OTA upgrade			Tbox reboot successfully after OTA upgrade			Android reboot successfully after OTA upgrade			Yocto reboot fail after OTA upgrade			Triggering system rollback	
		slot a	slot b		slot a	slot b		slot a	slot b		slot a	slot b		slot a	slot b		slot a	slot b		slot a	slot b
	priority	15	15		14	15		14	15		14	15		14	15		14	15		14	0
hypervisor	yocto_tries_remaining	3	3		1	3		1	1		1	1		1	1		1	1		1	0
	hyp_successful_boot	1	0		1	0		1	1		1	1		1	1		1	1		1	0
yocto	yocto_tries_remaining	3	3		1	3		1	3		1	3		1	3		1	0		1	0
	yocto_successful_boot	1	0		1	0		1	0		1	0		1	0		1	0		1	0
android	tries_remaining	7	7		1	7		1	7		1	7		1	1		1	1		1	0
	successful_boot	1	0		1	0		1	0		1	0		1	1		1	1		1	0
tbox	tbox_tries_remaining	3	3		1	3		1	3		1	1		1	1		1	1		1	0
	tbox_successful_boot	1	0		1	0		1	0		1	1		1	1		1	1		1	0

场景四：Hypervisor & Yocto & Tbox 启动成功，Android 启动失败（OTA 升级后重启）

Hypervisor (Nebula)	Linux SOS (Yocto)	Android UOS (Android)	Linux UOS (Tbox)
OK	OK	NG	OK

图 2-16. Hypervisor & Yocto & Android & Tbox 启动状态

各个阶段的 Boot Control 参数值变化如下所示：

		Normal boot			No reboot after OTA upgrade			Hypervisor reboot successfully after OTA upgrade			Yocto reboot successfully after OTA upgrade			Tbox reboot successfully after OTA upgrade			Android reboot fail after OTA upgrade			Triggering system rollback	
		slot a	slot b		slot a	slot b		slot a	slot b		slot a	slot b		slot a	slot b		slot a	slot b		slot a	slot b
	priority	15	15		14	15		14	15		14	15		14	15		14	15		14	0
hypervisor	yocto_tries_remaining	3	3		1	3		1	1		1	1		1	1		1	1		1	0
	hyp_successful_boot	1	0		1	0		1	1		1	1		1	1		1	1		1	0
yocto	yocto_tries_remaining	3	3		1	3		1	3		1	1		1	1		1	1		1	0
	yocto_successful_boot	1	0		1	0		1	0		1	1		1	1		1	1		1	0
android	tries_remaining	7	7		1	7		1	7		1	7		1	7		1	0		1	0
	successful_boot	1	0		1	0		1	0		1	0		1	0		1	0		1	0
tbox	tbox_tries_remaining	3	3		1	3		1	3		1	3		1	1		1	1		1	0
	tbox_successful_boot	1	0		1	0		1	0		1	0		1	1		1	1		1	0

场景五：Hypervisor & Yocto & Andorid 启动成功，Tbox 启动失败（OTA 升级后重启）

Hypervisor (Nebula)	Linux SOS (Yocto)	Android UOS (Android)	Linux UOS (Tbox)
OK	OK	OK	NG

图 2-17. Hypervisor & Yocto & Android & Tbox 启动状态

各个阶段的 Boot Control 参数值变化如下所示：

		Normal boot			No reboot after OTA upgrade			Hypervisor reboot successfully after OTA upgrade			Yocto reboot successfully after OTA upgrade			Android reboot successfully after OTA upgrade			Tbox reboot fail after OTA upgrade			Triggering system rollback	
		slot a	slot b		slot a	slot b		slot a	slot b		slot a	slot b		slot a	slot b		slot a	slot b		slot a	slot b
	priority	15	15		14	15		14	15		14	15		14	15		14	15		14	0
hypervisor	yocto_tries_remaining	3	3		1	3		1	1		1	1		1	1		1	1		1	0
	hyp_successful_boot	1	0		1	0		1	1		1	1		1	1		1	1		1	0
yocto	yocto_tries_remaining	3	3		1	3		1	3		1	1		1	1		1	1		1	0
	yocto_successful_boot	1	0		1	0		1	0		1	1		1	1		1	1		1	0
android	tries_remaining	7	7		1	7		1	7		1	7		1	1		1	1		1	0
	successful_boot	1	0		1	0		1	0		1	0		1	1		1	1		1	0
tbox	tbox_tries_remaining	3	3		1	3		1	3		1	3		1	0		1	0		1	0
	tbox_successful_boot	1	0		1	0		1	0		1	0		1	0		1	0		1	0

2.10.4 系统回滚测试用例

2.10.4.1 Yocto OS 启动失败

测试环境

- 1. The PC has Python 3 environment
- 2. Flash base load in DUT

3. Get OTA package from OTA owner

测试步骤

1. Device connection yocto adb environment
2. Execute the upgrade script: `python3 hypervisor_update.py --file otapackage_full.zip > update.txt 2>&1`
3. Check whether the upgrade is completed successfully, update.txt contains:
[INFO:update_attempter_android.cc(600)] **Update successfully applied, waiting to reboot.**
4. Execute in Yocto adb environment:
 - adb shell
 - blockdev --setrw /dev/disk/by-partlabel/yocto-boot_b
 - dd if=/dev/urandom of=/dev/disk/by-partlabel/yocto-boot_b bs=1M count=1
 - reboot

注:

- `hypervisor_update.py`: Update script
- `otapackage_full.zip`: OTA package, unlimited package name
- `> update.txt 2>&1`: Store the upgrade log in update.txt in the current directory

预期结果

1. Update should be completed, and `update.txt` prints the following log: [INFO:update_attempter_android.cc(600)] **Update successfully applied, waiting to reboot.**
2. The system triggers the rollback mechanism and boots to the HomeScreen
3. Setting-version should be the same as before the upgrade
Note: If the test upgrade fails, please provide the update log and uart log in CR.

2.10.4.2 Android OS 启动失败

测试环境

1. The PC has Python 3 environment
2. Flash base load in DUT
3. Get OTA package from OTA owner

测试步骤

1. Device connection yocto adb environment
2. Execute the upgrade script: `python3 hypervisor_update.py --file otapackage_full.zip > update.txt 2>&1`
3. Check whether the upgrade is completed successfully, update.txt contains:
[INFO:update_attempter_android.cc(600)] **Update successfully applied, waiting to reboot.**
4. Execute in Yocto adb environment:
 - adb shell
 - blockdev --setrw /dev/disk/by-partlabel/init_boot_b
 - dd if=/dev/urandom of=/dev/disk/by-partlabel/init_boot_b bs=1M count=1
 - reboot

注:

- `hypervisor_update.py`: Update script
- `otapackage_full.zip`: OTA package, unlimited package name
- `> update.txt 2>&1`: Store the upgrade log in update.txt in the current directory

预期结果

1. Update should be completed, and **update.txt** prints the following log: [INFO:update_attempter_android.cc(600)]
Update successfully applied, waiting to reboot.
2. The system triggers the rollback mechanism and boots to the HomeScreen
3. Setting-version should be the same as before the upgrade
Note: If the test upgrade fails, please provide the update log and uart log in CR

2.10.4.3 Tbox OS 启动失败

测试环境

1. The PC has Python 3 environment
2. Flash base load in DUT
3. Get OTA package from OTA owner

测试步骤

1. Device connection yocto adb environment
2. Execute the upgrade script: python3 **hypervisor_update.py** --file **otapackage_full.zip** > **update.txt 2>&1**
3. Check whether the upgrade is completed successfully, update.txt contains:
[INFO:update_attempter_android.cc(600)] **Update successfully applied, waiting to reboot.**
4. Execute in Yocto adb environment:
 - adb shell
 - blockdev --setrw /dev/disk/by-partlabel/boot_uos_tbox_b
 - dd if=/dev/urandom of=/dev/disk/by-partlabel/boot_uos_tbox_b bs=1M count=1
 - reboot

注:

- **hypervisor_update.py**: Update script
- **otapackage_full.zip**: OTA package, unlimited package name
- **> update.txt 2>&1**: Store the upgrade log in update.txt in the current directory

预期结果

1. Update should be completed, and **update.txt** prints the following log: [INFO:update_attempter_android.cc(600)]
Update successfully applied, waiting to reboot.
2. The system triggers the rollback mechanism and boots to the HomeScreen
3. Setting-version should be the same as before the upgrade
Note: If the test upgrade fails, please provide the update log and uart log in CR.

2.11 断电升级保护

2.11.1 升级状态信息

由于设备在 OTA 升级过程中存在异常断电导致升级中断的场景，为了保证设备在 OTA 升级过程中断电重启后能从中断点继续完成升级，系统需要实时保存升级状态信息，Hypervisor(L+L+A) 整体集中升级方案默认将升级状态信息保存在 yocto 的 `mnt/vendor/ota_prefs/prefs` 目录，存储的信息如图 2-18 所示：

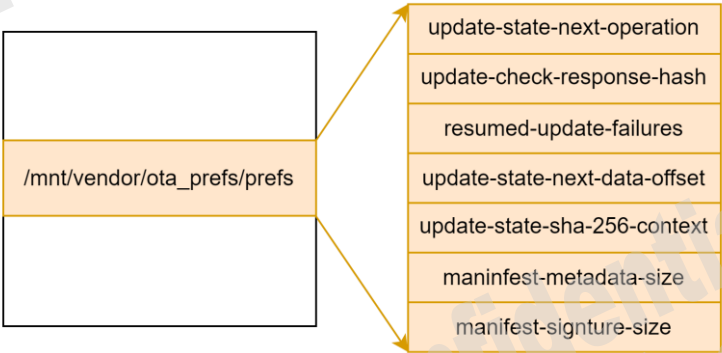


图 2-18. 升级状态信息

`prefs` 目录下各个升级状态信息的说明如表 2-5 所示：

表 2-5. 升级状态信息说明

Prefs 名称	Prefs 值	描述
kPrefsUpdateStateNextOperation	update-state-next-operation	记录下一次需要操作的 operation
kPrefsUpdateCheckResponseHash	update-check-response-hash	记录上一次升级包的 hash 值
kPrefsResumedUpdateFailures	resumed-update-failures	记录升级被中断的次数，默认一次升级最多被中断 10 次，如果超过 10 次，系统将强制全部重新升级
kPrefsUpdateStateNextDataOffset	update-state-next-data-offset	记录下一次从升级包内下载数据的偏移地址
kPrefsUpdateStateSHA256Context	update-state-sha-256-context	记录升级状态信息的 hash 值
kPrefsManifestMetadataSize	manifest-metadata-size	记录 metadata 数据 size
kPrefsManifestSignatureSize	manifest-signature-size	记录签名数据的 size

2.11.2 Resume 升级检查流程

OTA 升级前，系统会去判断当前升级是一次 new update 还是 resume update，其检测的信息来源于 prefs 目录内保存的信息，检测流程如图 2-19 所示：

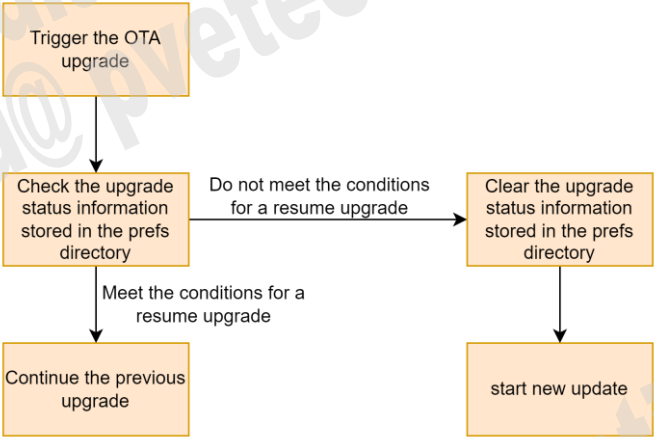


图 2-19. Hypervisor resume update 流程

其中检查 prefs 目录内存储的信息是否满足 resume 升级条件的详细流程如图 2-20 所示：

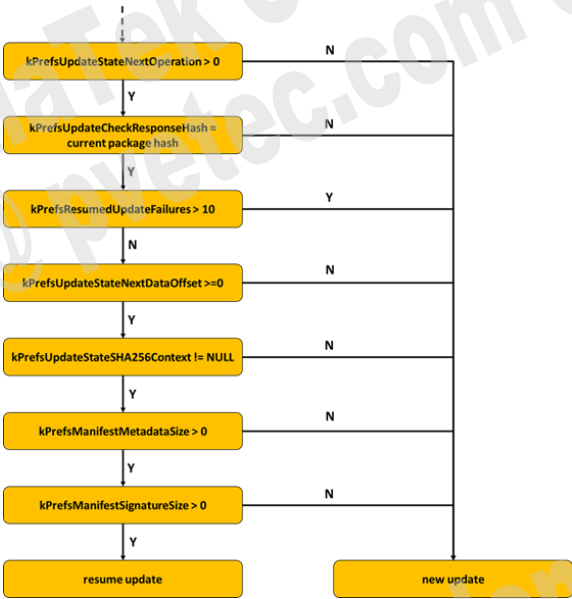


图 2-20. Hypervisor check resume update 流程

从图 2-20 可知，只有当 prefs 信息满足所有条件时，系统才会认得此次升级为 resume update；如果有任意条件不满足，系统都将全部重新开始升级并清除之前保存的升级状态信息。

2.12 Hypervisor OTA API 说明

MediaTek 在 Yocto SOS 端提供如下 OTA API 供客户使用。

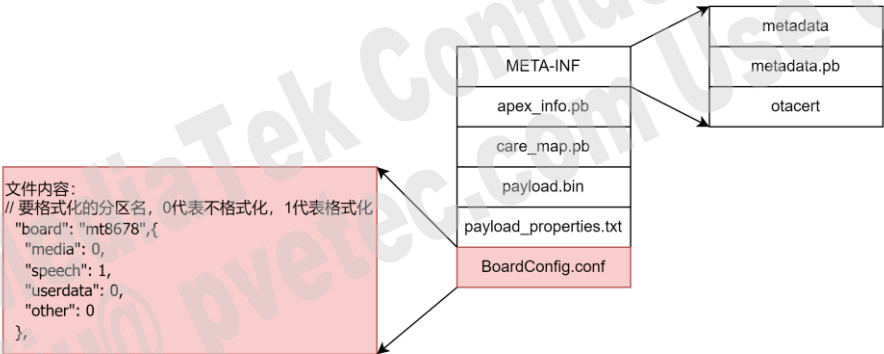
2.12.1 启动升级 (Normal Mode)

Normal mode 下提供两个启动升级的接口，第一个接口如下：

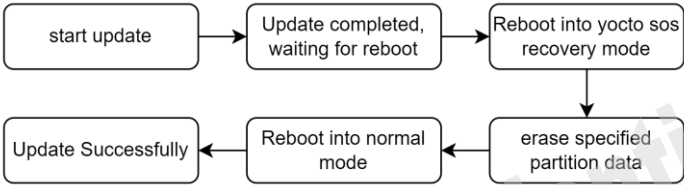
名称	描述
函数名称	StartUpdate
函数原型	bool StartUpdate (const char* update_package, bool is_format, bool should_resume)
函数定义	src/apps/atom-base/progs/bootctrl-u/ota-interface/files/ota_interface.h
函数实现	src/apps/atom-base/progs/bootctrl-u/ota-interface/files//ota_interface.cpp
函数功能	启动升级，并确认 OTA 升级完成后是否需要进行 format 分区操作和是否需要删除 ota_prefs 目录
函数返回值	True：启动升级成功 False：启动升级失败
传入参数	
update_package	参数说明：升级包路径 示例输入：const char* update_package = "/data/otapackage.zip";
is_format	参数说明：OTA 升级后是否需要进行 format 分区操作 True: 需要 format 分区; False: 不需要 format 分区 示例输入：bool is_format = true;
should_resume	参数说明：根据该参数判断升级前是否需要删除 ota_prefs 目录 True: 升级前不会删除 ota_prefs 目录，系统会判断当前升级是否需要从中断处继续; False: 升级前会删除 ota_prefs 目录，系统每次升级都会全部重新开始 示例输入：bool is_format = true;

注：

- 1. StartUpdate 函数是一个阻塞函数
- 2. 如果传入的 is_format 参数值为 true，用户需要保证升级包内有新增 BoardConfig.conf 文件 (BoardConfig.conf 文件需要用户自行添加，MTK 不提供添加流程)，用于标记本次需要擦除的分区信息，最终升级包格式如下：



3. 如果传入的 is_format 参数值为 false, 用户只需要按照章节 2.4 如何编译 Hypervisor (L+L+A) OTA 升级包 编译升级包
4. 由于擦除分区必须在 yocto (sos) recovery mode 下进行, 因此当 is_format 参数值为 true 时, OTA 升级完整流程如下:

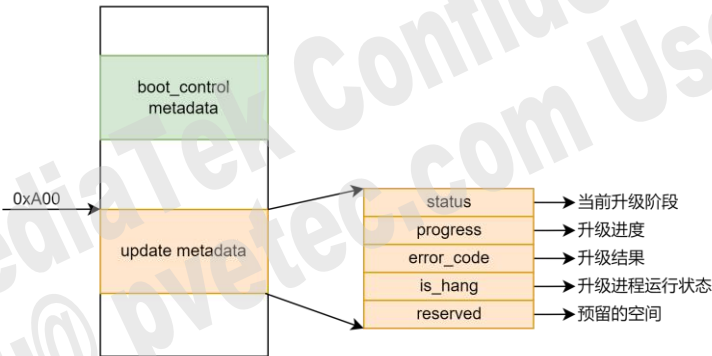


第二个接口如下:

名称	描述
函数名称	StartUpdate
函数原型	bool StartUpdate (const char* update_package)
函数定义	src/apps/atom-base/progs/bootctrl-u/ota-interface/files/ota_interface.h
函数实现	src/apps/atom-base/progs/bootctrl-u/ota-interface/files/ota_interface.cpp
函数功能	启动升级
函数返回值	True: 启动升级成功 False: 启动升级失败
传入参数	
update_package	参数说明: 升级包路径 示例输入: const char* update_package = "/data/otapackage.zip";

2.12.2 获取升级进度 (Normal Mode)

8668 30S (L+L+A) 在 misc 分区内新增如下结构体用于存储升级状态信息:



结构体定义: src/apps/atom-base/progs/bootctrl-u/files/hardware/mediatek/bootctrl/bootctrl_multi_os.h

名称	描述
函数名称	GetStatusAndProgress
函数原型	std::vector<std::string> GetStatusAndProgress()
函数定义	src/apps/atom-base/progs/bootctrl-u/ota-interface/files/ota_interface.h
函数实现	src/apps/atom-base/progs/bootctrl-u/ota-interface/files/ota_interface.h
函数功能	获取当前升级阶段、升级进度、升级结果
函数返回值	std::vector<std::string> update_metadata 示例输出: status=UPDATE_STATUS_UPDATED_NEED_REBOOT progress=1.000000 error_code=ErrorCode::kSuccess

注:

- 1. Status 的取值参考 Android source code: /system/update_engine/update_status_utils.cc
- 2. Error Code 的取值参考 Android source code: /system/update_engine/common/error_code.h

2.12.3 切换 Boot Slot (Normal Mode)

名称	描述
函数名称	SwitchSlot
函数原型	bool SwitchSlot (int64_t slot)
函数定义	src/apps/atom-base/progs/bootctrl-u/ota-interface/files/ota_interface.h
函数实现	src/apps/atom-base/progs/bootctrl-u/ota-interface/files/ota_interface.cpp
函数功能	切换系统启动插槽

名称	描述
函数返回值	True：切换成功 False：切换失败
传入参数	
slot	参数说明： Slot: 需要设定为系统下次启动的 boot slot，即 active slot; slot_a: 0; slot_b: 1 示例输入： int64_t slot = 1;

2.12.4 获取当前启动 Slot (Normal Mode)

名称	描述
函数名称	GetCurrentSlot
函数原型	int GetCurrentSlot ()
函数定义	src/apps/atom-base/progs/bootctrl-u/ota-interface/files/ota_interface.h
函数实现	src/apps/atom-base/progs/bootctrl-u/ota-interface/files/ota_interface.cpp
函数功能	获取系统当前启动 slot
函数返回值	0：boot slot_a 1：boot slot_b

2.12.5 中止升级 (Normal Mode)

名称	描述
函数名称	CancelCurrentUpdate
函数原型	bool CancelCurrentUpdate ()
函数定义	src/apps/atom-base/progs/bootctrl-u/ota-interface/files/ota_interface.h
函数实现	src/apps/atom-base/progs/bootctrl-u/ota-interface/files/ota_interface.cpp
函数功能	中止本次正在执行的升级进程
函数返回值	True：中止成功 False：中止失败

2.12.6 Hypervisor OTA API 使用步骤

Step1:

在<module-name>.bb 内添加 ota-interface lib 的 depend

```
DEPENDS += "ota-interface"
```

Step2:

在 Makefile.am 内添加如下 lib 的 link:

```
bin_LDADD += -lota_interface \
-lupdate_engine_boot_control -lpayload_consumer -lpayload_extent_ranges -
lpayload_extent_utils \
-lcow_operation_convert -llz4patch -lzucchini -lbootctrl -lmtkbsg -lchrome -lbrillo -
lbrillo_stream -lxz \
-lpuffpatch -lbrotli -lbootloader_message -lssl -lverity_tree -lcrypto -lotautil -lbz2 -
lziparchive -lbspatch\
-lsnapshot -lsnapshot_cow -lbrotli -ldm -lsnapshot_snapuserd -lfs_mgr -lcutils_u -
lcutils_sockets -lfs_avb -lavb -llp \
-lsparse -lfstab -lxt4_utils -lgz -lglz4 -lfsencrypt -lgsi -llogwrap_u -lgprotoc -
lprotobuf_cpp_lite -lchrome \
-latomic -lfmtlib -levent_updateengine -lbase_u -llog_u
```

Step3:

Source Code 内添加头文件 dm_interface.h

```
#include "ota_interface.h"
```

Step4:

在 Source Code 中调用接口

注:

MediaTek Demo Sample Code: src/apps/atom-base/progs/bootctrl-u/ota-interface /files/demo_main.cpp

附件一 附加条款

您(或您所代表的公司或其他法律主体, 以下合称「您」)是否得取得、下载与使用本文件及其相关信息皆应以您接受本附加条款为先决要件。您对于本文件之使用、取得或下载即表示您已接受本附加条款并同意受本附加条款之拘束。若您不同意受本附加条款之拘束, 您将不得使用、取得或下载本文件并应立即删除或毁弃所有本文件之副本。

本文件含有联发科技股份有限公司及其关联公司(以下合称「联发科技」)或其授权人之机密信息及专有信息, 仅供您为本文件所描述之联发科技芯片组作内部使用而不得用于其他任何目的(包括但不限于确认或提供支持任何对联发科技、其供货商及/或其直接或间接客户所提潜在专利侵权主张的证据)。禁止未经授权使用或揭露本文件及其中所含信息。您同意赔偿联发科技因您未经授权使用或揭露本文件及其中所含信息之一部或全部导致联发科技所受之任何损失或损害。

联发科技及其授权人保有本文件的所有权及组成所有权之各项权利且未针对任何知识产权授权(无论明示或默示、透过禁反言原则或其他方式)。联发科技得随时变更本文件内容而无须另行通知。联发科技无须承担与使用或信赖本文件相关或因使用或信赖本文件致生之任何责任, 包括但不限于间接损害或附带损害赔偿責任。

本文件及联发科技所提供关于本文件之任何其他材料、信息或技术支持, 均以现况交付为准, 联发科技不负任何明示、默示、法定或其他形式之担保责任, 特别是适销性、不侵权、针对特定用途之适用性、完整性或正确性之担保责任或任何由贸易惯例或交易、履行过程所生之担保责任。对于联发科技为符合您所提规格或遵循特定标准或标准组织的要求所作之交付物, 联发科技亦不应承担任何责任。

于未对前述条款造成限制之情形下, 联发科技不对其产品任何特定用途之适用性承担任何保证或担保责任, 亦不承担任何因产品、电路或软件之应用或使用致生之赔偿责任。您同意您应单独承担关于整合您的产品与联发科技产品所涉及之设计、验证与测试之所有责任, 并确保前述整合产品符合相关标准及任何安全性要求或其他要件。

本附加条款及所有与本附加条款或本文件相关之行为应以台湾法律管辖、解释与阐明, 不适用冲突法原则。